

0.101. Une lettre qui tire — un mur arrête le tir

Le 0.100, et un mur de X en colonne 12. Avant chaque pas, le tir regarde sa case d'arrivée avec `lire()` : si elle n'est pas vide, il s'écrase et disparaît ; le X reste.

1. Ce qu'il faut comprendre

C'est le 0.100, avec une chose en plus : un mur. Neuf X, debout, en colonne 12, des lignes 4 à 12. Le tir s'y écrase : il disparaît, et le mur, lui, reste.

Le mur : une boucle `for` (le 0.5) pose un X sur chaque ligne de 4 à 12 : `1 < 13` s'arrête **avant** 13. Le A s'y arrête déjà tout seul : depuis le 0.99, il ne va que sur une case **vide**.

Ce qui est nouveau ici : le tir regarde où il va. Comme le A (le 0.82) : on calcule l'arrivée, on la **regarde**, et seulement après on décide. Trois cas, dans cet ordre :

1. Hors de l'écran (`nx >= 20 || ny >= 17`) : il disparaît, comme avant. **2. L'arrivée n'est pas vide** (`lire(nx, ny) != 0`) : il **s'écrase** : `vole = 0`, et on ne le redessine pas. Sa case a déjà été effacée ; le X, sur la case d'arrivée, n'est pas touché. **3. Sinon**, la case est vide : il y va.

Pourquoi cet ordre ? `lire(20, 8)` n'a pas de sens : la colonne 20 n'existe pas. On vérifie donc d'abord qu'on est dans l'écran, **puis** on regarde la case. `else if` garantit qu'un seul des trois cas se fait.

Le départ aussi : collé au mur, le A n'a pas de place pour lancer le tir. `lire(nx, ny) == 0` s'ajoute au test du départ : si la case d'à côté est un X, rien ne part.

Déroulons : A en (2, 8), regard à droite. Bouton A : départ en (3, 8). Puis (4, 8)... (11, 8). Pas suivant : l'arrivée est (12, 8), `lire(12, 8)` rend le X, pas 0 : le tir s'écrase. (11, 8) est vide, le X est toujours en (12, 8).

Essaie : ajoute un deuxième mur, couché cette fois : `poser(c, 14, ALPHABET[23])` dans une boucle sur `c`.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <lire>` → [0.68.4. La fonction lire\(\) — lire une case](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **903 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	85 o	← la console, à l'allumage – la mise en route de la console
chaque_minuteur()	124 o	← main() – une fonction fournie par le langage
main()	448 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← image() – attend l'image suivante
AttendreVBlank	16 o	← démarrage – attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireTexte – attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), lire(), effacer(), texte() – trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer() – efface des cases
EcrireTexte	10 o	← texte() – écrit un texte, lettre par lettre
CopierTuiles	49 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l'écran
LireManette	37 o	← bouton() – lit les huit boutons
Tuiles	48 o	← CopierTuiles – 3 lettres sur 43
"."	1 o	← texte() – un texte gravé
"."	1 o	← texte() – un texte gravé

Lettres gravées : AX.

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

poser()	x 3	38 o sur place, dans main() + AdresseCase (10 o)
texte()	x 2	36 o sur place, dans main() + AdresseCase (10 o), EcrireTexte (10 o)
lire()	x 3	33 o sur place, dans main() + AdresseCase (10 o)
effacer()	x 2	30 o sur place, dans main() + AdresseCase (10 o), EffacerCases (12 o)
bouton()	x 5	25 o sur place, dans main() + LireManette (37 o)
image()	x 1	3 o sur place, dans main() + AttendreImage (28 o)
images()	x 1	2 o sur place, dans chaque_minuteur()

Tes fonctions appelées :

chaque_minuteur()	x 2	21 o sur place, dans main()
-------------------	-----	-----------------------------

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- UNE LETTRE QUI TIRE : un mur arrête le tir ----
// Le 0.100, et un mur de X au milieu de l'écran. Le tir s'y arrête.
// ALPHABET[23] : le X.
```

```
#include <poser>      // pose une tuile sur une case du fond
#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A
#include <chaque>      // répond 1 toutes les n millisecondes
#include <bouton>      // lit un bouton de la manette
#include <lire>        // lit la tuile posée sur une case
#include <effacer>     // efface des cases, ou tout le fond
#include <texte>       // écrit un texte à l'écran
```

2. UNE VARIABLE — AX

```
uint8_t ax = 2;          // le A : sa colonne...
```

3. UNE VARIABLE — AY

```
uint8_t ay = 8;          // ...et sa ligne
```

4. UNE VARIABLE — DX

```
uint8_t dx = 1;          // le sens où regarde le A (le 0.100)
```

5. UNE VARIABLE — DY

```
uint8_t dy = 0;
```

6. UNE VARIABLE — NX

```
uint8_t nx = 0;           // une case d'ARRIVÉE, calculée avant de bouger
```

7. UNE VARIABLE — NY

```
uint8_t ny = 0;
```

8. UNE VARIABLE — TX

```
uint8_t tx = 0;           // le tir : sa place...
```

9. UNE VARIABLE — TY

```
uint8_t ty = 0;
```

10. UNE VARIABLE — TDX

```
uint8_t tdx = 0;          // ...et son sens
```

11. UNE VARIABLE — TDY

```
uint8_t tdy = 0;
```

12. UNE VARIABLE — VOLE

```
uint8_t vole = 0;         // 1 = le tir est en vol ; 0 = pas de tir
```

13. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // ---- NOUVEAU : un mur de X, debout, en colonne 12, des lignes 4 à 12 ----
    for (uint8_t l = 4; l < 13; l++) {    // l vaut 4, 5, 6... jusqu'à 12 : 9 X
        poser(12, l, ALPHABET[23]);
    }
    poser(ax, ay, ALPHABET[0]);

    while (true) {
        image();

        // ---- le A bouge et retient où il regarde (le 0.100) ----
        // Il ne va que sur une case vide : le mur l'arrête déjà.
        if (chaque(150)) {
            nx = ax;
            ny = ay;
            if (bouton(DROITE)) {
                nx = ax + 1;
                dx = 1;
                dy = 0;
            } else if (bouton(GAUCHE)) {
                nx = ax - 1;
                dx = -1;
                dy = 0;
            } else if (bouton(BAS)) {
                ny = ay + 1;
                dx = 0;
                dy = 1;
            } else if (bouton(HAUT)) {
                ny = ay - 1;
                dx = 0;
                dy = -1;
            }
            if (nx < 20 && ny < 17) {
                if (lire(nx, ny) == 0) {
                    effacer(ax, ay, 1);
                    ax = nx;
                    ay = ny;
                    poser(ax, ay, ALPHABET[0]);
                }
            }
        }

        // ---- A lance le tir ----
        if (bouton(A) && vole == 0) {
            nx = ax + dx;
            ny = ay + dy;
            // NOUVEAU : la case de départ doit aussi être VIDE.
            // Collé au mur, le A ne tire pas : le tir n'a pas de place.
            if (nx < 20 && ny < 17 && lire(nx, ny) == 0) {
                vole = 1;
                tx = nx;
                ty = ny;
                tdx = dx;
                tdy = dy;
                texte(tx, ty, ".");
            }
        }

        // ---- le tir avance ; il regarde sa case d'arrivée AVANT d'y aller ----
        if (chaque(100)) {
            if (vole == 1) {
                nx = tx + tdx;
                ny = ty + tdy;
                effacer(tx, ty, 1);
            }
        }
    }
}
```

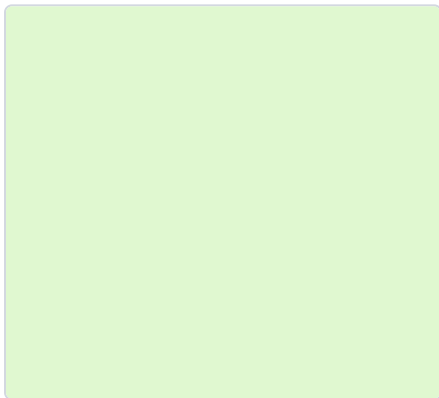
```

if (nx >= 20 || ny >= 17) {
    vole = 0; // 1. hors de l'écran : il disparaît
} else if (lire(nx, ny) != 0) {
    vole = 0; // 2. NOUVEAU : l'arrivée n'est pas vide
              // (un X) : il s'écrase, et disparaît.
              // Le X, lui, reste : on ne l'efface pas.
} else {
    tx = nx; // 3. l'arrivée est vide : il y va
    ty = ny;
    texte(tx, ty, ".");
}
// L'ordre compte : d'abord « hors de l'écran ? », car lire(20, 8)
// n'a pas de sens : la case n'existe pas.
}
}
}
}

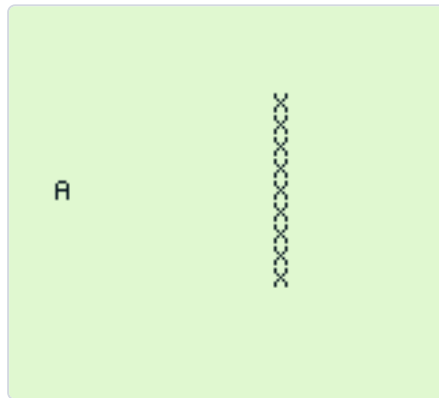
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



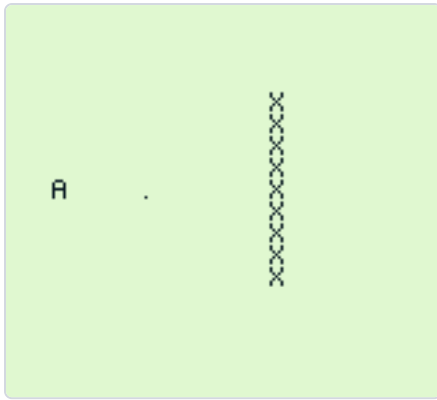
la toute première image — la console vient de démarrer — image 1



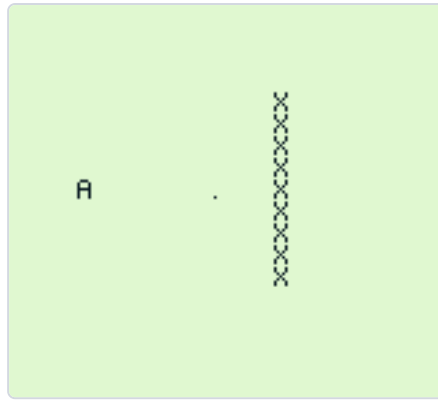
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

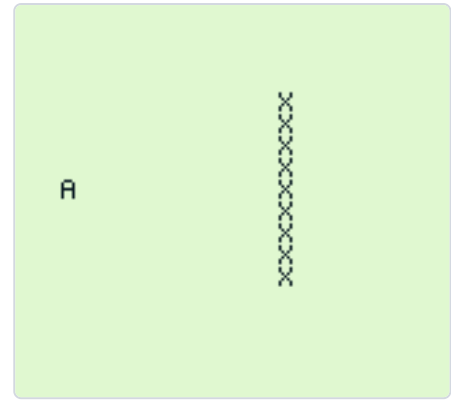
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



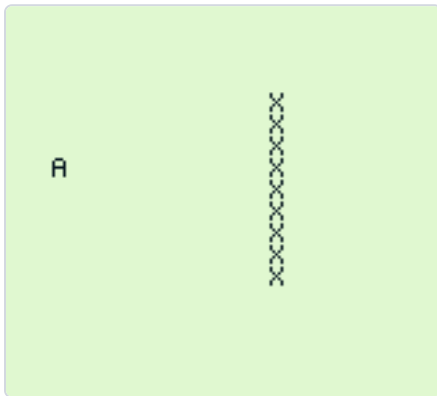
A — l'écran a changé



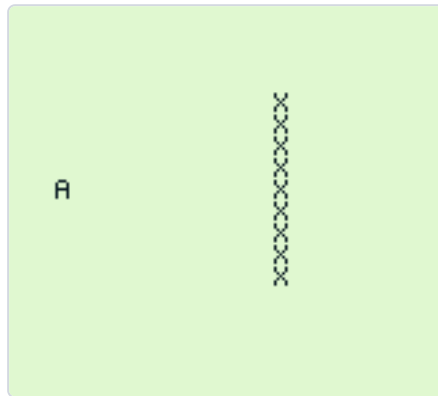
DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran a changé



BAS — l'écran a changé

5. Ce qu'on doit voir

Un mur de X en colonne 12. Le tir s'y écrase et disparaît, le mur reste. Collé au mur, le A ne peut pas tirer vers lui.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ un mur de 9 X en colonne 12, lignes 4 à 12
- ✓ le tir va jusqu'en colonne 11, puis s'écrase ; le X reste (le plus loin : colonne 11)
- ✓ le A s'arrête contre le mur, en colonne 11
- ✓ collé au mur, le A ne peut pas tirer

Ensuite : **0.101.1. Une lettre qui tire — un mur arrête les tirs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++